

COMMENCIS R&D

BugSTAiR v2

Bug Localization Research & Development
Project

-Eliz Payaslı, Kemal Arda Adışen



R&D at Commencis — Internship Journey

commencis

1

Onboarding

Structured learning: Udemy courses on LLM Mastery, Full-Stack Development with Ollama, and Generative AI.

2

Hands-on practice

Built RAG chatbots from scratch to internalize retrieval-augmented generation before applying it to a real research problem.

3

Research phase

Surveyed how bug localization is solved today: classical IR (BM25), learning-to-rank, deep learning, and the datasets the field evaluates on.

4

Applied research

Adapted our own pipeline to a real published benchmark (IQLoc) and iterated toward beating it.

What Is Bug Localization?

commencis

● Goal & Input

Given a bug report (natural-language text: title, description, stack trace fragments) and a codebase, predict which source files contain the fault.

● IR & Ranking

Formulated as an information-retrieval / ranking problem: rank every candidate file in the repository by likelihood of being the fix location.

● Evaluation Metrics

Evaluated the same way search engines are: Mean Reciprocal Rank (MRR), Mean Average Precision (MAP), Hit@k (is the true file in the top k?).

● Strategic Value

Sits upstream of automated program repair and developer triage, which is a better ranking directly shortens time-to-fix.

Why It's Hard?

commencis

1

Vocabulary mismatch

Bug reports use natural language; code uses identifiers, symbols, and domain jargon. Lexical overlap alone is weak.

2

Scale

Real repositories hold thousands of files. The true fix location is a needle in a large, noisy haystack.

3

Noisy reports

Reports mix signal (error messages, class names) with boilerplate (steps-to-reproduce, environment dumps) that actively hurts lexical search.

BugSTAIR v2 — A Second Generation of an Existing Idea

commencis

Why our system is called “BugSTAIR v2”

- BugSTAIR (2021, TÜBİTAK-funded, Mustafa Ersahin et al.): the original system, combining a source-code index and a separate commit-history index for bug localization.
- This project builds our own version of that same idea, commit-history as a second retrieval signal, but fused via Reciprocal Rank Fusion (RRF) instead of a literal candidate-set union.
- We also start from a stronger retrieval baseline than IQLoc's own: hybrid BM25 + embedding retrieval (RRF-fused), not IQLoc's pure-BM25 first stage.
- “BugSTAIR v2” = this project's own system throughout this deck and the SOTA comparison table, which is a second generation of the original idea, with new additions of our own.

Why Bench4BL / IQLoc?

commencis

- We chose Bench4BL as our benchmark because it's purpose-built for bug localization specifically, not bug fixing, not classification.
- "Localization" here means: given a bug report, find the source file(s) actually changed to fix it, using the real historical commit as ground truth.
- IQLoc publishes a corrected, extended version of Bench4BL with a trained cross-encoder + keyword reformulation on top of BM25, a real, citable published result to measure ourselves against, not just an internal baseline.
- This is the single benchmark this entire project's headline results are measured on.

How Big Are These Datasets, Really?

Scale of Bench4BL, IQLoc, and what we actually processed

Dataset	Projects / Repos	Total Bug Instances	What we processed
Bench4BL	53 open-source Java projects	4,418 bug instances	Used for baseline representation testing
IQLoc (raw)	Same 53 projects	7,483 raw records	Starting point before resolution filtering
IQLoc (resolvable)	Same 53 projects	7,399 instances	After confirming files/commits actually resolve
IQLoc (strict, our target)	Same 53 projects	7,091 instances	Every ground-truth file verified resolvable
Our full-scale runs	Same 53 projects	7,090 / 7,091 processed	99.99% coverage — 1 instance excluded

IQLoc is a corrected, extended version of Bench4BL's raw data — same underlying 53 projects, with resolution filtering applied to guarantee every ground-truth file can actually be verified against the repository history.



Camel / CAMEL-21407

camel-dynamic-router component: unsubscribe by control message does not work

Details

Type:



Bug **a**

Priority:



Minor

Affects Version/s:

4.8.0 **d**

Component/s:

camel-core

Labels:

None

Estimated Complexity:

Unknown

Status:

RESOLVED **b**

Resolution:

Fixed **c**

Fix Version/s:

4.8.2, 4.9.0 **e**

Description

A user reported a problem with unsubscribing from dynamic router by using a control message, and it wasn't working. However, unsubscribing with headers works as expected.

Issue Links

links to



GitHub Pull Request #16128 **f**

Fig. 5. An example bug report from JIRA.

Table 9

Performance of IQLoc for different classes of bug reports.

(a) Time-wise Split					
Model	MAP	MRR	HIT@1	HIT@5	HIT@10
ST					
Baseline Elasticsearch	0.488	0.513	0.443	0.594	0.686
IQLoc	0.565	0.601	0.535	0.698	0.730
PE					
Baseline Elasticsearch	0.599	0.621	0.505	0.768	0.874
IQLoc	0.689	0.737	0.674	0.811	0.863
NL					
Baseline Elasticsearch	0.442	0.467	0.350	0.612	0.709
IQLoc	0.460	0.487	0.382	0.626	0.720
(b) Random Split					
Model	MAP	MRR	HIT@1	HIT@5	HIT@10
ST					
Baseline Elasticsearch	0.492	0.513	0.421	0.637	0.721
IQLoc	0.561	0.592	0.503	0.708	0.755
PE					
Baseline Elasticsearch	0.582	0.632	0.542	0.759	0.819
IQLoc	0.655	0.687	0.602	0.771	0.843
NL					
Baseline Elasticsearch	0.423	0.454	0.353	0.592	0.680
IQLoc	0.432	0.456	0.350	0.596	0.686

Table 2

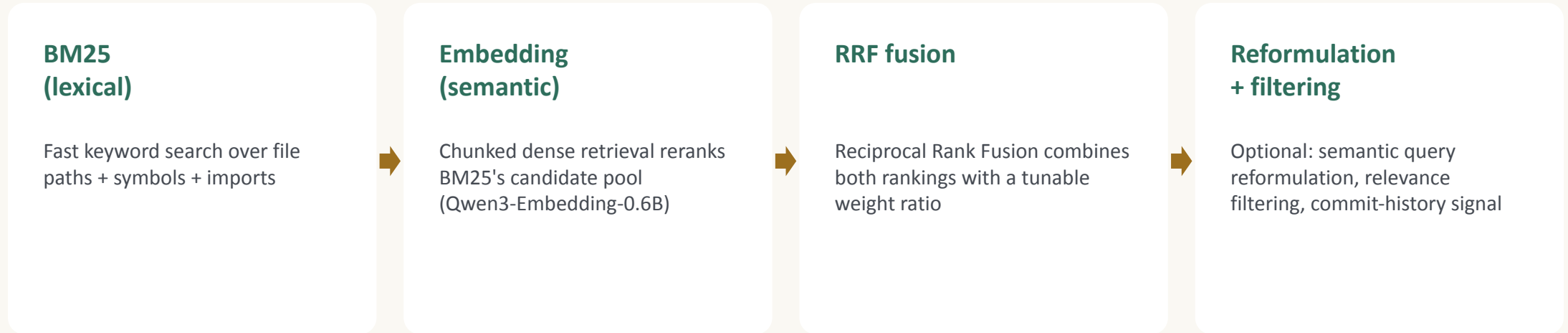
Bench4BL dataset summary.

Project	Subject-systems	Bug reports
Apache	13	4503
Jbosss	8	1505
Spring	25	3451
Old Subject	5	558
Total	51	10,017

Our Architecture

commencis

Hybrid retrieval: lexical + semantic signals, fused, then optionally refined



- Every stage is swappable and independently tested: retriever, embedding model, reformulation mode, relevance-filter mode all sweep-tested before adoption.
- Evaluated first at n=23 (fast iteration), then always validated at full population before being trusted, small samples have repeatedly misled us.

Step 2 Explained: Reformulation, Reranking & the LLM Call

What each piece actually does, on top of Step 1's retrieval

Commit-History Signal

A third retrieval signal alongside BM25 and embeddings: files touched by commits whose messages resemble the bug report get a ranking boost. Fused via RRF, not a simple union.

Relevance Filtering

An LLM (or cosine similarity) judges each retrieved candidate chunk: is this plausibly relevant to the bug? Candidates judged "not relevant" get pushed down, not deleted — recall stays intact.

Semantic Reformulation

Keywords are extracted from the candidates judged relevant (via EmbedRank/MMR), appended to the original bug report, and the SAME retriever runs a second pass with this enriched query.

Reranking

The second-pass ranking (using the reformulated query) becomes the final output — a genuine second look, not just reordering the same list.

The LLM Call Job

Chunks are judged in batches (not all 100-200 candidates in one call) — this bounds how much a single LLM response needs to say, which matters a lot for reasoning models with long, variable-length chain-of-thought.

MN5 HPC Environment

- MareNostrum 5 (Barcelona Supercomputing Center) — GPU-accelerated H100 partition, used for every embedding-model and local-LLM run in this project.
- Full-population evaluation requires re-ranking ~7,000 bug instances with a GPU-hosted embedding model, infeasible on a laptop at this scale.
- Access secured through Commencis' HPC allocation (account ehpc680), which is a real institutional resource, not a personal or trial account.
- Large evaluations run as Slurm array jobs (up to 350 shards in parallel), with atomic per-shard writes so partial progress is never lost if capacity drops.
- GPU availability fluctuates significantly with overall cluster load, swinging between 0 and 350+ concurrent jobs on the same day, entirely outside our control, and something we learned to work around by prioritizing and re-queuing jobs live.

BugSTAiR v2 — Progress Timeline



Version	Stage	Scale	MRR	Hit@1	Hit@5	Hit@10	MAP
STEP 1 — Retrieval only (varying representation, embedding, weight/k)							
v2.1	BM25 alone (original representation)	sample	0.231	13.0%	39.1%	43.5%	0.168
v2.2	Embedding alone (Qwen3)	sample	0.561	39.1%	78.3%	87.0%	0.391
v2.3	Dense alone (full-corpus, Qwen3, no BM25)	sample	0.622	43.5%	87.0%	91.3%	0.449
v2.4	Hybrid RRF (old BM25 + Qwen3, best weight/k)	sample	0.582	43.5%	78.3%	87.0%	0.395
v2.5	BM25 refinement — refined BM25 alone	sample	0.503	34.8%	65.2%	78.3%	0.359
v2.6	Hybrid RRF (refined BM25 + Qwen3, best weight/k) — BEST	sample	0.7145	56.5%	82.6%	95.7%	0.499
STEP 2 — Reformulation, added gradually on top of Step 1's winner							
2a	Retriever baseline (refined BM25 + Qwen3 + commit-history)	sample	0.525	26.1%	87.0%	95.7%	0.422
2b	+ relevance filtering (LLM)	sample	0.629	39.1%	87.0%	95.7%	0.494
2c	+ semantic reformulation (final)	sample	0.625	47.8%	82.6%	95.7%	0.466
STEP 3 — Full population (confirmed)							
vF.1	Old BM25, full pipeline, CONFIRMED	full (n=7,090)	0.518	40.3%	65.6%	73.8%	0.416
vF.2	Refined BM25 + Qwen3, full pipeline, CONFIRMED	full (n=7,090)	0.5557	48.8%	76.5%	84.5%	0.454

Step 2 uses the clean, validated cross-check (refined BM25 + OpenAI); the qwen3-coder version of this same test is still being debugged for a token-truncation bug.

SOTA Comparison



System	Benchmark	MRR	MAP	Hit@1
BugSTAIR v2 (ours), CONFIRMED	IQLoc, n=7,090/7,091	0.5557	0.4539	48.8%
BRain	Bench4BL (own split)	0.571	0.537	—
IQLoc (time-wise split, published)	IQLoc dataset	0.553	0.520	46.6%
IQLoc (random split, published)	IQLoc dataset	0.520	0.493	42.3%
RLocator	Bench4BL	0.511	0.478	41.9%
Blizzard	Bench4BL	0.510	0.480	41.0%
BLUIR	Bench4BL	0.504	0.470	39.3%
Elasticsearch (plain BM25)	Bench4BL	0.486	0.457	38.9%

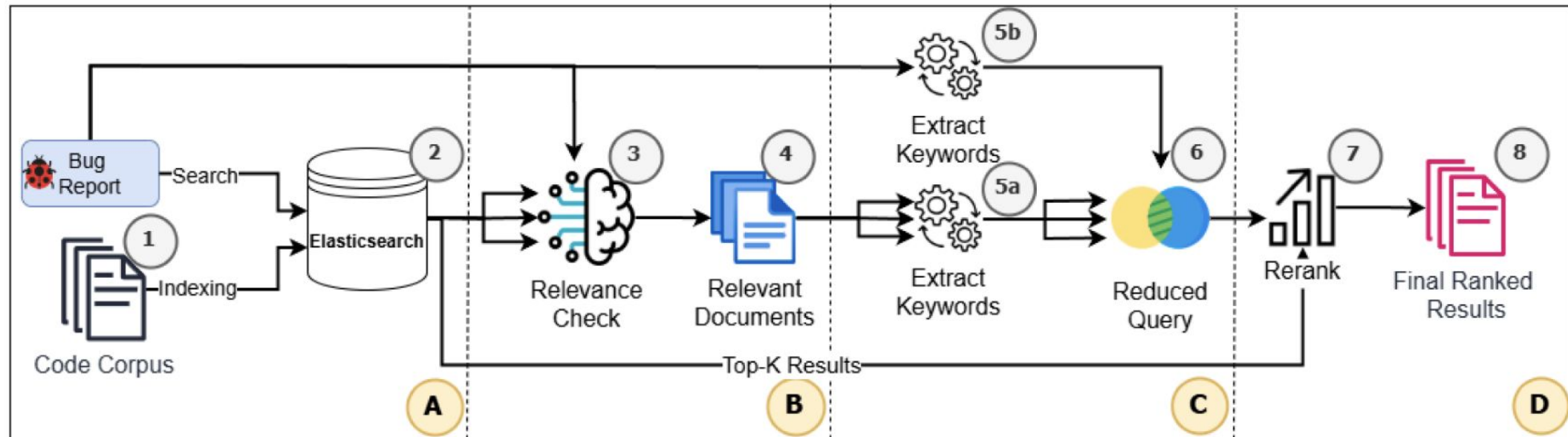


Fig. 4. Schematic diagram of *IQLoc*: (A) Indexing & retrieval of documents, (B) Check relevancy, (C) Query reformulation, (D) Bug-localization.

From onboarding to large-scale evaluation

A simple view of how the internship progressed.



Each stage built on the previous one — from learning concepts to evaluating thousands of bug reports.

What I learned before the research problem

Concepts from onboarding that transferred directly into Bugstairv2.

Generative AI

LLMs & prompts

RAG & embeddings

retrieve + ask

LangChain / LangGraph

agents & tools

Open-source LLMs

Llama / Mistral

Applied in Bugstairv2

retrieval + LLM

Why this mattered

Retrieval, embeddings, RAG-style candidate selection, and LLM second-look thinking mapped directly onto the Bugstairv2 pipeline.

Research Skills & R&D Mindset I Developed

How Commencis R&D changed the way I approach a problem

commencis

Academic Research Mindset

Papers → hypotheses → controlled experiments

Learned to treat an engineering problem like a research question: review related work, understand how prior systems were evaluated, design controlled experiments, and validate before claiming a result.

Information Retrieval, in Practice

Lexical + semantic + hybrid ranking

Learned how retrieval really works — keyword matching, embedding-based semantic search, combining signals, reranking. Key lesson: an LLM can only reason well over candidates retrieval already shortlisted.

Reading Research Like a Researcher

IQLoc → new ideas → fair evaluation

Studying a published system taught me to read past the headline number — inspect the evaluation methodology, understand what a metric measures, and treat prior work as inspiration for new experiments.

Large-Scale, Offline Research Engineering

MareNostrum 5 → reproducible validation at scale

Scaling from dozens of cases to thousands meant working in an HPC environment with no live internet access — everything had to be prepared and made self-contained in advance.

An R&D Mindset

Iterate, document, collaborate, learn from evidence

Ask "why?" before "how?". Treat a failed experiment as real evidence. Present progress regularly. Use AI tools as research assistants that accelerate discovery — never as unquestioned sources.

Commencis R&D taught me how to turn an AI problem into a structured research process — from literature and hypothesis to large-scale evidence.

The scientific target became stronger

The pipeline was reused — this was not a restart.



Better datasets sharpened the question — they did not throw away earlier work.

How the system improved step by step

A research timeline: different rows use different datasets / sample sizes, so read the N column before comparing.

Milestone	What changed	Dataset / N	MRR	MAP	Hit@1	Hit@5	Hit@10	Outcome
v2.1	Path-only BM25	BeetleBox · N=24	.074	—	.042	.125	.125	Retrieval bottleneck found
v2.2	Path + symbols + imports	BeetleBox · N=24	.162	—	.083	.250	.333	Richer code signals retained
v2.3	Frozen lexical baseline	Bench4BL · N=4,621	.416	.337	.291	.554	.672	Full BM25 baseline
v2.4	Controlled BM25	Controlled · N=55	.511	.442	.400	.636	.800	Lexical control
v2.5	Jina dense retrieval	Controlled · N=55	.655	.505	.564	.764	.836	Dense retrieval selected
v2.6	Hybrid BM25 + Jina	Controlled · N=55	.697	.563	.564	.836	.909	Hybrid looked strongest
v2.7	Final BM25	Author-final · N=7,023	.403	.324	.282	.539	.648	Lexical parent
v2.8	Final Jina	Author-final · N=7,023	.519	.418	.396	.669	.755	Dense parent
v2.9	Final Hybrid RRF	Author-final · N=7,023	.554	.454	.430	.705	.793	Frozen retrieval H
v2.10	Qwen H+SR pilot	Pilot · N=24	.615	.515	.500	.750	.917	Selected for full run
v2.11	Full H+SR validation	Author-final · N=7,023	TBD	TBD	TBD	TBD	TBD	IN PROCESS

How to read it: early rows show learning and controlled experiments; the final author cohort is N=7,023. The last row stays TBD until the full H+SR run is frozen.

From keyword search to a smarter second look

Final evaluation set: N = 7,023.

BM25

keyword matching

Hit@1 .282
Hit@5 .539
Hit@10 .648
MRR .403
MAP .324

Jina

meaning-based search

Hit@1 .396
Hit@5 .669
Hit@10 .755
MRR .519
MAP .418

Hybrid

combine both

Hit@1 .430
Hit@5 .705
Hit@10 .793
MRR .554
MAP .454

H+SR

LLM second look

Full metrics TBD
Status IN PROCESS

N=24 selected the method

A pilot to choose the full-run approach — not final performance.

Method	Idea	Hit@1	Hit@5	Hit@10	MRR	MAP
H	Hybrid baseline	.375	.583	.667	.495	.438
H+RF/QR	Improve the query	.417	.750	.792	.543	.447
H+SR	Rerank candidates	.500	.750	.917	.615	.515
Full	Query + re-search + rerank	.458	.708	.792	.563	.471

N=24 selected the method.
N=7,023 determines the final result.

Secondary robustness check with another LLM: 12/24 · metrics TBD

Where Bugstairv2 stands today

Selected published baselines + our current completed result. Published rows are reference points, not identical-population comparisons.

System	Benchmark / setting	N	MRR	MAP	Hit@1	Hit@5	Hit@10	Mechanism / status
Bugstairv2 H+SR	Author-final primary	7,023	TBD	TBD	TBD	TBD	TBD	Qwen semantic reranking · IN PROCESS
Bugstairv2 Hybrid	Author-final primary	7,023	.554	.454	.430	.705	.793	BM25 + Jina fusion · COMPLETE
IQLoc	Published time-wise split	1,501	.553	.520	.466	.669	.735	BM25 + trained reranking
IQLoc	Published random split	1,497	.520	.493	.423	.647	.721	BM25 + trained reranking
RLocator	Published random split	—	.511	.478	.419	.652	.726	RL-based reranking
Blizzard	Published random split	—	.510	.480	.410	.648	.735	IR / query reformulation
BLUiR	Published random split	—	.504	.470	.393	.655	.745	Structured IR
Elasticsearch	Published random split	—	.486	.457	.389	.618	.703	Plain BM25 baseline

Technical skills, research habits, R&D teamwork

Personal takeaways from building Bugstairv2.

Technical

- RAG / retrieval pipelines
- Embeddings & vector search
- Lexical vs meaning search
- LLM reranking
- Scaling experiments on HPC

Research

- Controlled method comparisons
- Sample-size limits
- Reproducibility
- Validate before scaling
- Careful claims on interim results

R&D / Professional

- Pair work on one problem
- Frequent supervisor feedback
- Document decisions & adapt
- Explain results clearly
- Iterate when better data arrives

Final H+SR evaluation is still running.

Thank You for Listening!

